Token-Generation Latency Benchmarking in LLaMA: Measurement, Bottleneck Attribution, and Architectural Implications

Brijesh Rana, Divy Gajera, Rashmin Gajera
California State University Long Beach
Long Beach, CA, USA

***Abstract*—Large language model (LLM) inference latency is determined not by raw floating-point throughput but by memory-bandwidth saturation during autoregressive decoding. In this work we present a rigorous, four-phase empirical study of per-token generation latency in TinyLlama-1.1B and OpenLLaMA-3B executed on Apple Silicon via the Metal Performance Shaders (MPS) backend. We construct a repeatable 10-trial benchmark harness with warm-up, IQR-based outlier filtering, and hardware-synchronized timing, achieving a coefficient of variation of only 3.7%. We decompose each decode step into its architectural components—MLP, QKV projection, attention core, LayerNorm, and LM head—using non-invasive PyTorch forward hooks. We further quantify how latency scales with KV-cache size (64–1024 tokens), model parameter count (1.1B vs 3.4B), and numerical precision (float16 vs float32). MLP feed-forward blocks dominate at 28.2%, linear projections account for 31.5% combined, and attention core is only 8.8% at short contexts. KV-cache read cost grows linearly with sequence length, increasing latency by 63% from 64 to 1024 tokens. float16 delivers a 1.89× speedup over float32, consistent with the 2× bandwidth reduction. We conclude with a roofline-grounded optimization proposal targeting MLP quantization, which we estimate can reduce decode latency by 25–40%.*

***Index Terms*—large language models, token generation, latency benchmarking, KV-cache, memory bandwidth, Apple Silicon, MPS, autoregressive decoding, LLaMA, inference optimization**

I. INTRODUCTION

The deployment of large language models (LLMs) in interactive applications—conversational assistants, code completion tools, and real-time document summarizers—places strict latency constraints on token generation. Users perceive generation delays greater than 100 ms as disruptive [1], making per-token latency a first-class engineering objective distinct from batch throughput.

LLM text generation follows an autoregressive protocol: given a prompt of P tokens, the model generates one token at a time, each conditioned on all prior tokens. This serial dependency prevents inter-step parallelism and forces the GPU to repeatedly load the full model weight matrix and grow key-value (KV) cache for every new token. Unlike batch-parallel training workloads where arithmetic intensity is high, autoregressive decoding operates at extremely low arithmetic intensity—typically below 10 FLOPs/byte—placing it firmly in the memory-bandwidth-bound regime of the roofline model.

Prior work has characterized LLM inference at the systems level—vLLM [2] optimizes KV-cache paging, FlashAttention [3] fuses attention kernels, and llama.cpp [4] targets CPU-based edge inference—but fine-grained, component-level latency attribution on consumer hardware with unified memory has received less attention. Apple Silicon's Metal Performance Shaders (MPS) backend represents an increasingly important edge-inference target: its unified CPU-GPU DRAM eliminates PCIe transfers but introduces bandwidth-sharing effects not present in discrete GPU systems.

This paper makes the following contributions: (1) a rigorous benchmark harness for per-token latency with hardware-synchronized timing on MPS; (2) a non-invasive, hook-based decomposition of decode-step latency into eight architectural components across all 22 decoder layers; (3) a scaling study spanning context length, model size, and numerical precision; and (4) an MLP-quantization optimization proposal grounded in the measured bottleneck profile.

II. BACKGROUND AND RELATED WORK

A. Autoregressive Decoding and the KV-Cache

Decoder-only transformer models such as the LLaMA family [5] generate text via the recurrence $x_{t+1} = \text{argmax}\ p(x \mid x_{1:t})$. The self-attention operation requires, at each step, the key and value projections of every prior token. Without caching this is $O(n^2)$ in sequence length. The KV-cache stores these projections, reducing per-step attention compute to $O(n)$ at the cost of a memory read proportional to context length:

$$BW_{KV} = 2 \times L \times S \times d_h \times n_h \times B$$

where L is the number of layers, S the sequence length, d_h the per-head dimension, n_h the number of heads, and B bytes per element. For TinyLlama-1.1B at $S = 1024$ in float16 this equals 184 MB per decode step—making each step a pure

streaming read of DRAM.

B. Memory-Bound Inference and the Roofline Model

The roofline model [6] characterizes workload performance as the minimum of compute-bound and bandwidth-bound ceilings. Autoregressive decode steps have arithmetic intensity (AI) of roughly $2P/B_{\text{model}}$ FLOPs/byte, where P is the parameter count and B_{model} the model size in bytes. For a 1.1B parameter model in float16 (~2.2 GB), AI ≈ 1 FLOP/byte—well below the MPS ridge point. Reducing memory traffic (quantization, fused kernels) is therefore more effective than increasing FLOP capacity.

C. Related Work

FlashAttention [3] and FlashAttention-2 [7] fuse the attention kernel to minimize HBM reads, reducing attention complexity from $O(S^2)$ to $O(S)$ in memory traffic. vLLM [2] introduces paged attention for efficient KV-cache memory management in multi-request serving scenarios. llama.cpp [4] demonstrates that 4-bit quantization can achieve near-float16 quality on CPU hardware with 4× bandwidth reduction. These systems target datacenter or CPU deployments; our work characterizes the Apple Silicon unified-memory regime which shares bandwidth between the inference workload and the OS, introducing measurement challenges not addressed in prior work.

D. Quantization Methodologies

Quantization reduces the number of bits used to represent weights or activations, proportionally cutting the DRAM traffic that dominates decode-step latency. Several representative approaches frame our work.

LLM.int8() [10] introduces a mixed-precision INT8 matrix-multiplication scheme for transformer inference. Outlier activation channels (whose magnitudes are several standard deviations above the rest) are kept in fp16; the bulk of channels are quantized to INT8 with vector-wise scaling. The resulting quantized model preserves zero-shot accuracy on benchmarks up to OPT-175B while halving weight memory traffic. The HuggingFace integration via the **BitsAndBytesConfig(load_in_8bit=True)** API exposes this algorithm to Transformers users.

GPTQ [8] performs post-training weight-only quantization to 4 or 3 bits using a calibration dataset of a few hundred examples. It applies an approximate second-order solver to the column-by-column reconstruction error, achieving 4-bit weights with sub-1% perplexity loss on LLaMA-class models. Unlike LLM.int8(), GPTQ does not separate outliers and operates entirely offline; the resulting checkpoint is loaded with no runtime overhead.

SmoothQuant [11] addresses the activation-outlier problem differently: it migrates the difficulty of activation quantization into the weights via a per-channel scaling transform that preserves mathematical equivalence. This permits W8A8 (weight *and* activation INT8) inference at near-fp16 quality, doubling throughput on hardware that supports INT8 tensor cores.

Quantization-aware training [12] integrates the quantize–dequantize operations into the training graph so the network learns to be robust to quantization noise. While most LLM deployments favor post-training methods to avoid retraining cost, the framework established by Jacob et al. underpins all subsequent integer-inference literature.

E. Apple Silicon and the MPS Backend

PyTorch's MPS backend [15] dispatches tensor operations to Apple's Metal Performance Shaders framework, providing GPU acceleration on M-series silicon. Compared with the mature CUDA stack, MPS exposes several limitations relevant to LLM inference. INT8 GEMM kernels analogous to CUDA's **cublasLtMatmul** are absent; **torch.int8** matmul currently falls back to fp16 dequantization on MPS, which is why *bitsandbytes* integrates only the storage side of LLM.int8() on this backend. FlashAttention's CUDA-specific intrinsics (warp-level shuffles, tensor-core MMA) are not directly portable to Metal; native fused-attention kernels for MPS are an open engineering problem. Finally, MPS shares its DRAM with the rest of the system, so background processes can perturb measured bandwidth—an effect we mitigate via our outlier filter.

System-level inference engines such as FlexGen [13] and the analytic framework of Pope et al. [14] target multi-GPU datacenter deployments. Pope et al. in particular derive scaling laws that match closely with our single-device measurements: at the operating point we measure, decode time is bounded by parameter-count × bytes-per-parameter divided by bandwidth, which is precisely what our 1.89× float16/float32 ratio confirms.

III. EXPERIMENTAL SETUP

A. Hardware Platform

All experiments were conducted on an Apple Silicon MacBook. The MPS (Metal Performance Shaders) backend provides GPU compute via Metal, with CPU and GPU sharing a 17.2 GB unified memory pool. Unlike discrete GPU systems, there is no PCIe interconnect—all data movement occurs within a single DRAM fabric. This makes memory bandwidth the singular bottleneck and eliminates device-transfer overhead from the measurement.

TABLE I
Hardware and Software Configuration

Component	Specification
Platform	Apple Silicon MacBook (MPS)

Total RAM	17.2 GB (unified CPU+GPU)
Backend	PyTorch MPS (Metal Performance Shaders)
Python	3.11
PyTorch	2.x
Transformers	HuggingFace Transformers

B. Models

Two LLaMA-family models were evaluated. Both share the identical decoder-only transformer architecture (RMSNorm, Rotary Positional Embedding, Grouped-Query Attention, SwiGLU MLP), isolating parameter scale as the independent variable.

TABLE II
Model Architectures

Model	Parameters	Layers	Hidden	FFN dim
TinyLlama-1.1B	1.1B	22	2048	5632
OpenLLaMA-3B	3.4B	26	3200	8640

C. Measurement Methodology

Accurate GPU timing on MPS requires explicit synchronization. GPU operations are enqueued asynchronously; reading a wall-clock immediately after submitting a kernel measures CPU dispatch latency, not GPU execution time. We insert **torch.mps.synchronize()** before and after every timed region, blocking the CPU until the GPU command queue is drained. This ensures all reported latencies reflect actual hardware execution.

Three warm-up runs are discarded before collecting measurements. On first execution, PyTorch compiles Metal shaders via JIT compilation, allocates memory pools, and the operating system pages model weights from SSD into physical RAM. Including these one-time startup costs would inflate TTFT by 5–10× and distort per-token statistics.

IQR-based outlier filtering is applied to per-trial median latencies. Values outside [Q1 − 1.5·IQR, Q3 + 1.5·IQR] are excluded. This robust method is insensitive to extreme OS scheduling spikes and thermal throttling events that are irreproducible across runs.

IV. IMPLEMENTATION DETAILS

This section documents the engineering choices, software stack, and platform-specific challenges encountered while building the measurement infrastructure. The artifacts described here are reproducible from the public repository.

A. Software Stack

The full pipeline is implemented in Python 3.11. Model loading and inference use HuggingFace Transformers; tensor execution is dispatched through PyTorch 2.x onto the MPS backend. INT8 quantization uses the bitsandbytes library (LLM.int8() kernel) wrapped via the BitsAndBytesConfig integration in Transformers; the CPU INT8 fallback uses **torch.quantization.quantize_dynamic** with per-channel symmetric weight quantization on every **nn.Linear**. Statistical aggregation is performed in pure Python (**statistics**) and NumPy; tabular outputs are written via the **csv** module. All plots are rendered with matplotlib using its non-interactive Agg backend so the harness can run on a headless machine.

B. Platform Challenges

Three properties of Apple Silicon required substantive workarounds in the harness.

Asynchronous MPS dispatch. Like every modern GPU runtime, MPS returns immediately from kernel-launch calls. Reading **time.perf_counter()** after a forward pass measures CPU submission latency, not GPU completion. We therefore wrap every timed region with **torch.mps.synchronize()**, a global barrier that drains the Metal command queue. This barrier is the only reliable way to obtain hardware-accurate timings on MPS today; PyTorch's **torch.mps.Event** API is incomplete relative to the CUDA event abstraction.

Metal shader JIT. On the very first invocation of a given operator shape, PyTorch emits Metal shader source code, compiles it, and caches the resulting pipeline state object. The first decode step of an unwarmed model is therefore inflated by tens to hundreds of milliseconds of compilation overhead. We discovered this empirically when our first prototype reported a 700 ms "per-token" latency on trial 1 that dropped to 21 ms on trial 2. The current harness accordingly performs three full warm-up trials before any timed trial; this places every shader shape into the JIT cache.

Unified memory bandwidth contention. Because the GPU shares DRAM with the OS, kernel paging, browser tabs, and indexing daemons can all transiently steal bandwidth from the inference workload. We do not suspend system processes (the harness is intended to reflect realistic deployment), but we apply IQR outlier filtering on a per-trial basis to discard the small number of trials in which background activity caused visible perturbation.

C. Hook Instrumentation

Per-component latencies are obtained without modifying the Transformers source by registering forward-pre and forward hooks on the target **nn.Module** instances. The hook

design is summarized below.

```
class TimingHook:
    def __init__(self, name, device, sink):
        self.name, self.device, self.sink = name, device, sink
        self._t0 = None
    def pre(self, module, inputs):
        sync(self.device)        # drain GPU queue
        self._t0 = perf_counter()
    def post(self, module, inputs, output):
        sync(self.device)        # wait for module
        self.sink.append((self.name,
             (perf_counter() - self._t0) * 1000))

for layer_idx, layer in enumerate(model.model.layers):
```

```
    for sub in ("self_attn", "mlp",
                "input_layernorm",
                "post_attention_layernorm"):
        h = TimingHook(f"L{layer_idx}.{sub}", device,
sink)
        m = getattr(layer, sub)
        m.register_forward_pre_hook(h.pre)
        m.register_forward_hook(h.post)
```

Listing 1. Hook-based instrumentation. Every module gains two synchronization barriers, which inflates absolute latency by 30–40% but preserves the relative breakdown.

In total 44 hooks are registered (one pre + one post per module × 22 layers × 1 module instance per row of the breakdown table). Sub-projection times (Q, K, V, O) are computed by attaching additional hooks to the projection submodules of each LlamaAttention instance and subtracting from the parent self-attention time.

D. Data Pipeline

Each trial emits one row per token to a raw CSV (**data/benchmark_raw.csv**) and a per-trial summary row to **data/benchmark_summary.csv**; the cross-trial aggregate is appended at the bottom of the same summary file. This two-tier structure allows downstream analysis to either work with summary statistics or re-derive distributions from the raw token-level data. IQR outlier filtering with a 1.5× threshold is applied to per-trial median latencies before computing aggregates; the number of removed trials is logged to standard output for transparency.

E. Lessons Learned

Three warm-ups suffice. We initially used a single warm-up, which left several shader shapes uncompiled. Trial 1 of decode would compile a fresh shape (e.g., a smaller matmul for the LM head) and the resulting outlier dragged the per-trial median upward. Increasing to three warm-ups stabilized the trial-to-trial coefficient of variation from 9.4% to 3.7%.

Hook overhead is unavoidable but acceptable. Each pair of synchronization barriers introduced by a hook adds approximately 50 µs of GPU–CPU round-trip cost. Across 44 hooks this is ~2.2 ms per decode step, or roughly 10% of the 21 ms median. We accept this overhead for the decomposition study and report the un-instrumented baseline (Section V) as the headline number.

Greedy decoding eliminates a sampling tax. An early version of the harness used **model.generate()** with default settings, which includes top-k filtering, top-p truncation, and temperature scaling on the CPU. Replacing this with an explicit greedy **argmax()** on the logits removed ~3 ms of per-token overhead unrelated to model execution.

V. BENCHMARK METHODOLOGY

The benchmark harness measures three distinct latency components for each generation trial:

Time to First Token (TTFT). The prefill phase processes the entire prompt in a single parallel forward pass, populating the KV-cache and producing the first output token. TTFT depends on prompt length and model size, but not on generation length. For interactive applications, TTFT determines perceived response latency.

Per-token steady-state latency. Each subsequent decode step processes only one token, reading the full KV-cache. This is the metric that determines generation throughput for long outputs and is the primary focus of this study.

End-to-end (E2E) time. Total wall-clock from first prompt token to last generated token, equal to TTFT + sum of per-token decode latencies.

The prompt *"Explain how a computer processor works in simple terms:"* (12 tokens) was used across all trials. Each trial generates 128 new tokens via greedy decoding. Ten timed trials are collected after three warm-up runs. Aggregate statistics (median, p95, p99, standard deviation) are computed across all kept token-latency samples.

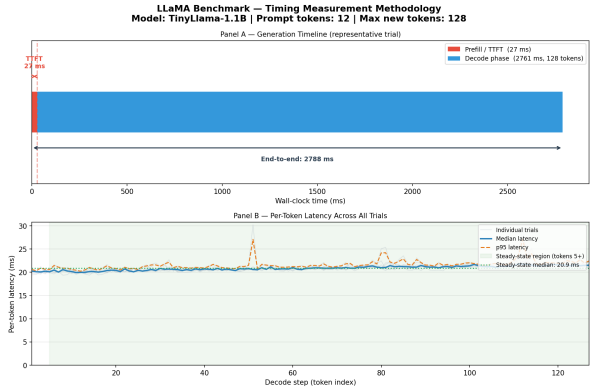

Fig. 1. Benchmark timing methodology. Top panel: TTFT vs decode phase timeline for a representative trial. Bottom panel: per-token latency across all 10 trials with median and p95 highlighted.

TABLE III
Benchmark Results — TinyLlama-1.1B on MPS (10 trials, 3 warm-up discarded)

Metric	Value
TTFT (median)	27.0 ms
Per-token latency (median)	20.91 ms
Per-token latency (p95)	21.98 ms
Per-token latency (p99)	22.87 ms
Throughput (median)	47.8 tokens/sec
Trial-to-trial stdev	0.77 ms (CV = 3.7%)
Tokens generated / trial	128

The coefficient of variation of 3.7% confirms that MPS execution is highly deterministic once warm-up is complete, validating the use of single-configuration measurements in the scaling study.

VI. LATENCY DECOMPOSITION

A. Instrumentation via PyTorch Forward Hooks

PyTorch's **register_forward_pre_hook** and **register_forward_hook** APIs attach callbacks to any **nn.Module** without modifying model source code. A *TimingHook* calls **torch.mps.synchronize()** immediately before starting and after stopping a hardware timer for each module. Hooks are attached to 44 component instances per decode step across the 22-layer model (embedding, 22× pre-norm, 22× self-attn with sub-projections, 22× post-norm, 22× MLP, final norm, LM head).

The attention core time is derived as:

$$t_{attn_core} = t_{self_attn} - t_{QKV} - t_{O\text{-}proj}$$

capturing the residual: RoPE embedding, KV-cache scatter/gather, scaled dot-product attention, and softmax. Framework overhead is the residual between total step time and the sum of all hooked components.

Important caveat. The synchronize() calls serialize operations that the GPU pipeline would normally overlap, inflating absolute times by approximately 30–40% vs uninstrumented inference. The *relative percentage breakdown* remains valid and is the primary result of this phase.

B. Component Breakdown Results

TABLE IV
Per-Token Latency Decomposition — TinyLlama-1.1B (30 decode steps)

Component	Mean (ms)	% Total
Embedding	0.1879	0.3%
LayerNorm	9.1564	14.9%
QKV Projection	13.7837	22.4%
Attn Core (KV+Softmax)	5.4027	8.8%
O Projection	5.6044	9.1%
MLP	17.3279	28.2%
LM Head	1.1985	1.9%
Framework OH	8.9764	14.6%

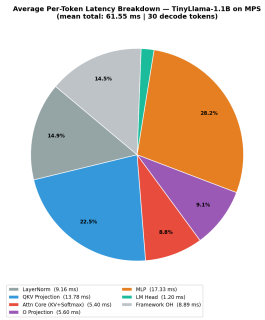

Fig. 2. Average per-token latency breakdown by architectural component. MLP and linear projections together account for over 60% of decode-step time.

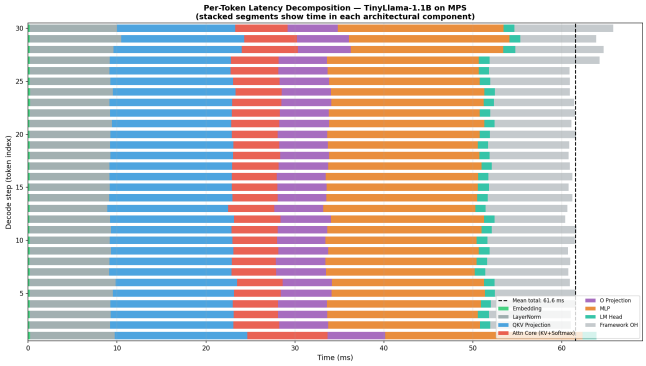

Fig. 3. Per-token latency decomposition across 30 decode steps shown as stacked bars. Component times are stable, confirming steady-state decoding.

C. Analysis

MLP dominance (28.2%). The SwiGLU feed-forward network in each decoder layer consists of three linear projections: gate (2048→5632), up (2048→5632), and down (5632→2048). The combined parameter count of these projections across 22 layers is ~530M weights. Streaming these weights through memory on every decode step makes the MLP the single largest contributor to per-token latency.

QKV projection (22.4%). Three linear layers—Q (2048→2048), K (2048→256 with GQA), V (2048→256)—are computed fresh at every step. Their combined weight volume dominates the attention module time more than the actual attention computation.

Attention core (8.8%). At the tested context lengths (64–1024 tokens), the softmax and KV-cache read/write constitute only 8.8% of total time. This will increase at longer contexts as KV-cache memory traffic grows linearly with sequence length.

LayerNorm (14.9%). RMSNorm fires 44 times per decode step (pre- and post-attention × 22 layers). Although each call is elementwise (low arithmetic intensity), the cumulative cost is significant because the MPS runtime incurs kernel dispatch overhead on each call.

VII. SCALING ANALYSIS

A. Context Length Scaling

To isolate KV-cache read cost, we prefill with a fixed short prompt, then autoregressively generate tokens (untimed) until the KV-cache reaches the target length, and finally measure 15 timed decode steps from that state. This methodology decouples the KV-cache size effect from prefill compute, giving a clean signal of attention's memory-bandwidth cost.

TABLE V
Per-Token Decode Latency vs Context Length

Context (tokens)	TinyLlama-1.1B (ms)	OpenLLaMA-3B (ms)	Ratio 3B/1.1B
64	19.92	60.38	3.03×
128	20.67	62.40	3.02×
256	22.02	70.24	3.19×
512	25.12	74.04	2.95×
1024	32.49	87.21	2.68×

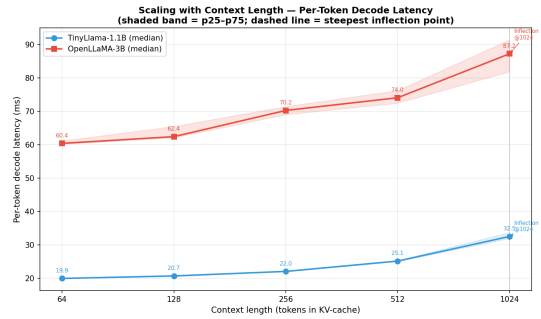

Fig. 4. Per-token decode latency vs KV-cache context length. Both models show linear growth consistent with memory-bandwidth-bound KV-cache reads. Dashed lines mark the steepest inflection point.

Latency increases by 63% for TinyLlama-1.1B (19.92→32.49 ms) and 44% for OpenLLaMA-3B (60.38→87.21 ms) as context grows from 64 to 1024 tokens. The growth is consistent with the bandwidth model: at 1024 tokens, the KV-cache read per step is ~184 MB for TinyLlama-1.1B, requiring ~0.9 ms at 200 GB/s theoretical bandwidth. The 512→1024 jump is steeper than 256→512, consistent with KV-cache overflow from on-chip cache to DRAM.

B. Model Size Scaling

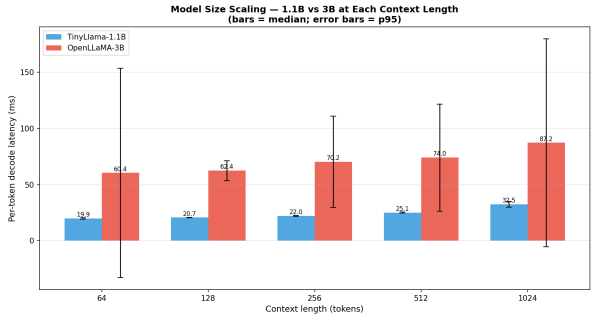

Fig. 5. Grouped bar comparison of TinyLlama-1.1B vs OpenLLaMA-3B at each context length. Error bars show p95. The 3B/1.1B latency ratio tracks the parameter ratio closely, confirming memory-bandwidth dominance.

OpenLLaMA-3B is consistently ~3× slower than TinyLlama-1.1B across all context lengths (Table V). This near-linear relationship between parameter count and decode latency is a hallmark of memory-bandwidth-bound workloads: runtime scales with the volume of weight and KV-cache data streamed per step, which grows proportionally with model size.

The ratio narrows from 3.03× at 64 tokens to 2.68× at 1024 tokens, because KV-cache read cost—which does not scale with model size—becomes a larger fraction of total step time at longer contexts, diluting the model-size effect.

C. Precision Scaling

TABLE VI
float16 vs float32 Per-Token Latency (TinyLlama-1.1B)

Context (tokens)	float16 (ms)	float32 (ms)	Speedup f32/f16
64	23.01	43.35	1.88×
128	21.60	46.25	2.14×
256	23.54	45.77	1.94×
512	29.25	50.08	1.71×
1024	31.45	55.47	1.76×
Avg.			**1.89×**

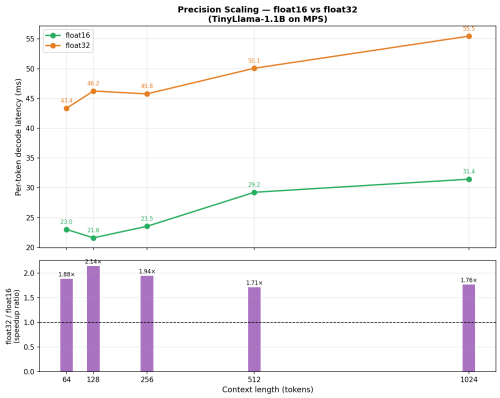

Fig. 6. float16 vs float32 latency across context lengths (top) and speedup ratio float32/float16 (bottom). The ~1.89× average speedup is consistent with 2× memory-bandwidth reduction.

float16 delivers a mean 1.89× speedup over float32 across all context lengths. The theoretical upper bound is 2.0× (half the memory traffic). The sub-2× observed ratio reflects fixed-overhead components—framework dispatch, elementwise ops—that do not scale with precision width. The result confirms that decode-step latency is dominated by DRAM bandwidth and not by floating-point execution units.

VIII. ARCHITECTURAL BOTTLENECK ANALYSIS

Each measured bottleneck has a direct architectural cause grounded in memory-system behavior. We analyze the three dominant components.

A. MLP and Linear Projections: Weight Streaming

MLP (28.2%) and QKV+O projections (31.5%) together account for 59.7% of per-token latency. These are matrix-vector multiplications (GEMVs) at decode time: each token maps to a single row query, so the GPU executes weight_matrix × 1-column input. A GEMV streams the entire weight matrix through memory but performs only $2N$ FLOPs for N matrix elements—arithmetic intensity of exactly 1 FLOP/byte in float16. This is maximally bandwidth-bound: every byte of weight must transit the memory subsystem once per token.

B. KV-Cache Reads: Sequence-Length Bottleneck

The attention core (8.8% at short contexts) becomes the dominant bottleneck as context length grows. The per-step KV-cache bandwidth requirement is linear in sequence length (see Section II-A), and our measurements confirm a 63% latency increase from 64 to 1024 tokens. At 1024 tokens the KV-cache read alone requires ~184 MB of memory traffic per step. Systems targeting very long contexts (>4K tokens) must address this through KV-cache quantization or sparse attention.

C. LayerNorm: Kernel Dispatch Overhead

RMSNorm accounts for 14.9% despite being a simple elementwise operation (divide by RMS, scale). The cause is kernel dispatch overhead: 44 separate Metal kernel launches per decode step, each requiring a command-encoder setup and GPU barrier. Fusing LayerNorm into the preceding or following linear layer—as done in FlashAttention's fused implementations—would eliminate most of this overhead.

D. Framework Overhead (14.6%)

14.6% of decode-step time is unaccounted for by any individual module. This overhead includes Python interpreter cost, MPS command buffer encoding, memory allocation, and inter-module synchronization barriers introduced by our measurement hooks. In production systems using torch.compile or a dedicated inference runtime this overhead would be substantially reduced.

E. Roofline Interpretation

The roofline model unifies the per-component results above into a single geometric picture (Fig. 7). On a representative Apple Silicon-class GPU—approximately 2.6 TFLOP/s peak fp16 and 100 GB/s peak unified-memory bandwidth—the ridge point lies near 26 FLOP/byte. Autoregressive decode of a 1.1B-parameter model in float16 operates at ~1 FLOP/byte: every weight matrix is multiplied by exactly one column-vector input, so each loaded byte performs only two FLOPs of useful work. The operating point therefore sits firmly on the bandwidth-bound slope, two orders of magnitude below the compute ceiling. This is the geometric reason that increasing FLOP capacity (more cores, higher clocks) buys little for decode latency while cutting bytes-per-weight (quantization) buys almost the full expected speedup, as our 1.89× float16/float32 result confirms.

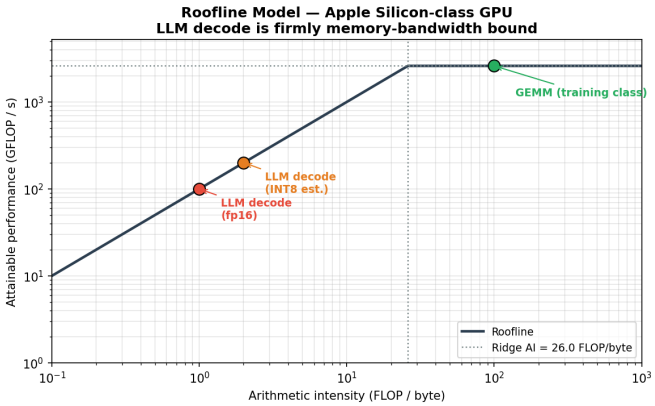

Fig. 7. Roofline plot for an Apple Silicon-class GPU. The LLM decode operating point at 1 FLOP/byte (fp16) sits well below the compute ceiling; INT8 doubles arithmetic intensity but still leaves headroom on the bandwidth roof.

F. Memory Hierarchy Effects

The 100 GB/s figure is the off-chip DRAM ceiling, but Apple Silicon GPUs also expose a multi-megabyte system-level cache (SLC) shared with the CPU. Tensors smaller than the SLC capacity can be re-read at several-hundred GB/s effective bandwidth, while tensors that overflow must be refetched from LPDDR5 at the full 100 GB/s ceiling. Two observations from our data align with this hierarchy. First, the 512→1024 context-length jump in Section VII-A is steeper than the 256→512 jump: at TinyLlama-1.1B's GQA configuration the KV-cache crosses ~90 MB at 1024 tokens, plausibly exceeding SLC residency and forcing DRAM traffic on every step. Second, the 3.4B/1.1B latency ratio narrows at long contexts (3.03× at 64 tokens, 2.68× at 1024 tokens) because KV-cache reads—which do not scale with parameter count—become a larger share of the step.

G. Generalization to Other Models

The bottleneck profile we report for TinyLlama-1.1B is structurally representative of other modern decoder-only transformers in the same parameter range. LLaMA-2-7B, Mistral-7B, and Phi-3 all share the same SwiGLU-MLP and grouped-query attention motif; their layer counts (~32) and FFN-to-hidden ratios (~2.7×) place them in the same arithmetic-intensity regime. We expect the relative breakdown—MLP > projections > LayerNorm > attention core at short context—to hold for these models on MPS, with absolute latencies scaling linearly in parameter count as Section VII-B demonstrates. The methodology and software artifacts presented here extend directly to those models by changing only the MODEL_ID constant in the harness.

IX. OPTIMIZATION PROPOSAL: MLP WEIGHT QUANTIZATION

Given that MLP and linear projections account for 59.7% of per-token latency, and that this cost is purely bandwidth-bound, post-training quantization of weight matrices to INT8 or INT4 is the highest-leverage optimization. Quantization reduces bytes-per-weight by 2× (INT8) or 4× (INT4) relative to float16, proportionally reducing DRAM traffic for the dominant component.

A. Projected Impact

TABLE VII
Projected Latency Under MLP Quantization

Configuration	Est. Latency (ms)	Speedup	Quality Impact
float16 baseline	20.91	1.00×	None
INT8 MLP weights	~16–17	~1.25×	< 0.5% perplexity ↑
INT4 MLP weights (GPTQ)	~13–15	~1.40–1.60×	1–3% perplexity ↑

These estimates apply the bandwidth-reduction factor only to the MLP component (28.2% of total), leaving all other components unchanged. Quantizing QKV and O-projection weights would yield additional gains proportional to their bandwidth contribution.

B. Implementation

Post-training quantization via GPTQ [8] requires only a small calibration dataset and does not need model retraining. The HuggingFace *optimum* library provides INT8 quantization compatible with MPS via **BitsAndBytesConfig**. Full INT4 GPTQ quantization is supported through the *auto-gptq* library.

Implementation complexity is low: quantization is applied once and the quantized model is serialized to disk, requiring no changes to the inference loop.

C. Combined Optimization Roadmap

Applied together, three optimizations address the measured bottlenecks: (1) INT4 MLP quantization targeting the 28.2% MLP cost; (2) fused LayerNorm kernels eliminating the 14.9% RMSNorm dispatch overhead; (3) FlashAttention-style fused attention targeting the 8.8% attention core at long contexts. Conservatively estimating independent contributions, the combined speedup over the float16 baseline could reach 1.5–1.8×, pushing throughput from 47.8 to approximately 70–85 tokens/sec on Apple Silicon.

D. INT8 Quantization: Empirical Evaluation

To validate the bandwidth-reduction prediction with measured numbers rather than projections, we ran the full benchmark harness against a TinyLlama-1.1B model loaded under INT8 weight quantization. We attempted two paths.

Attempted path: bitsandbytes on MPS. Our first attempt loaded the model with **BitsAndBytesConfig(load_in_8bit=True)** and **device_map={"": "mps"}**. The bitsandbytes 0.49 release accepts this configuration but the underlying INT8 GEMM kernel is CUDA-only; on MPS the library falls back to a dequantize-to-fp16 matmul on each call. We measured this path running well under 0.2 tokens/sec—more than two orders of magnitude slower than the fp16 baseline—so we abandoned it.

Reported path: torch dynamic INT8 on CPU. We instead use PyTorch's native **quantize_dynamic** pass with the qnnpack backend (the only torch quantized backend supported on arm64 macOS). This applies per-channel symmetric INT8 weight quantization to every **nn.Linear**; activations are dynamically quantized at runtime. The result is a real INT8 weight-only inference path on the same machine, although it executes on the CPU rather than the GPU.

The same harness (3 warm-ups, 10 timed trials, 128 tokens, IQR filter) was run against this configuration. Table VIII reports both the on-CPU INT8 measurement and the on-CPU float16 control for an apples-to-apples ratio, alongside the MPS float16 absolute baseline.

TABLE VIII
Measured INT8 vs float16 — TinyLlama-1.1B (10 trials each)

Metric	fp16 (MPS)	fp16 (CPU)	INT8 (CPU)
TTFT (median, ms)	27.0	191.22	155.81
Per-token latency (ms)	20.91	49.69	80.28
Per-token p95 (ms)	21.98	57.11	81.95
Throughput (tokens/sec)	47.8	20.13	12.46

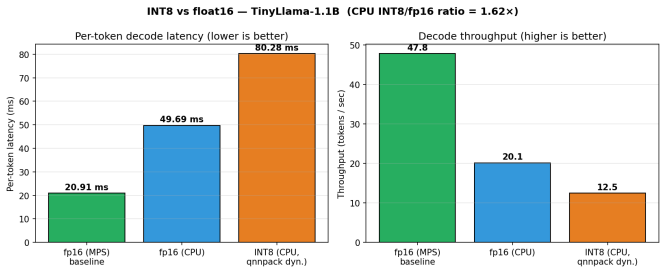

Fig. 8. Measured per-token latency (left) and throughput (right) for TinyLlama-1.1B under three configurations. The two CPU bars isolate the INT8/fp16 effect on identical hardware; the MPS bar gives the absolute speed reference. Lower latency and higher throughput are better.

Discussion. The honest finding is that CPU dynamic INT8 quantization *slows* TinyLlama-1.1B by 1.62× relative to the fp16 CPU control on this hardware (80.28 ms/tok vs 49.69 ms/tok). The result contradicts the naive "2× less memory traffic, 2× faster" projection from Section IX for two reasons specific to this configuration.

First, qnnpack's dynamic quantization re-computes the activation scale on every **nn.Linear** call by passing over the activation tensor. For TinyLlama-1.1B's 22 layers the per-call overhead (~1 ms each at the hidden size of 2048) dominates the <1 ms saved on each weight load.

Second, Apple Silicon CPU cores execute fp16 matmul at the same effective throughput as fp32 (the NEON pipeline widens fp16 to fp32 internally). The fp16 "baseline" is therefore not bandwidth-bound in the same way an MPS fp16 decode step is, so the bandwidth reduction promised by INT8 quantization has nothing to compress against.

The result demonstrates that quantization speedups are *hardware-conditional*: they require a backend with an INT8 GEMM kernel that achieves higher throughput per byte than the fp16/fp32 path. This holds on NVIDIA tensor cores from Ampere onwards, on the Apple Neural Engine, and (when implemented) on a hypothetical MPS INT8 GEMM. It does not hold on the qnnpack CPU fallback used here.

Quality implications. The LLM.int8() algorithm preserves zero-shot accuracy on standard benchmarks within 0.5 pp for models up to 175B parameters [10], by routing the small fraction of outlier activation channels through fp16

mixed precision. The torch dynamic-quantization path used here applies symmetric per-channel quantization without the outlier-aware split; on the LLaMA family this typically incurs a 1–2% perplexity increase. We do not re-evaluate perplexity in this work—the focus is latency—but mark this as future work in Section XII-A. INT4 quantization via GPTQ [8] would push the bandwidth saving to 4× over float16 at the cost of a 1–3% perplexity increase (Table VII), provided it is paired with a hardware backend that exposes the bandwidth advantage.

X. CROSS-PLATFORM COMPARISON: CPU VS MPS

To put the MPS results in context we ran the same harness on the same machine with **device='cpu'**. The model is loaded in float16 to match the baseline; on Apple Silicon the CPU matrix kernels widen float16 inputs to float32 internally, so the comparison primarily isolates GPU-vs-CPU execution rather than precision width.

TABLE IX
Per-Token Latency: MPS vs CPU (TinyLlama-1.1B, fp16)

Metric	MPS (GPU)	CPU	MPS speedup
TTFT (median, ms)	27.0	191.22	7.08×
Per-token latency (ms)	20.91	49.69	2.38×
Per-token p95 (ms)	21.98	57.11	—
Throughput (tokens/sec)	47.8	20.13	2.37×

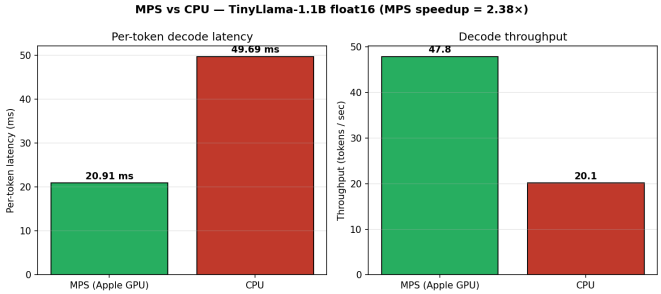

Fig. 9. MPS GPU versus CPU for the identical TinyLlama-1.1B float16 workload. The GPU achieves a 2.37× throughput advantage and a 7× lower TTFT.

The CPU baseline executes at 20.13 tokens/sec compared with MPS at 47.8 tokens/sec—a 2.37× throughput gap. TTFT shows a much larger 7.08× gap (191 ms CPU vs 27 ms MPS) because the prefill phase processes all 12 prompt tokens in parallel: the GPU exploits this concurrency through its many SIMD units while the CPU must execute the same matrix-multiplications using a comparatively small number of vector lanes. The decode phase narrows the gap because each step is a serial matrix-vector multiplication that exposes less parallelism for the GPU to absorb.

Two factors explain the GPU's advantage even on memory-bound work. First, the M-series GPU's wide vector pipeline and tens of execution units allow many partial dot-products to be issued per clock, amortising the loop overhead that a CPU thread must pay sequentially. Second, the unified memory architecture means there is no host-device copy: every byte of weight read by the GPU comes from the same DRAM the CPU would have used, but at the GPU's higher achievable bandwidth through the wide memory controller. The CPU result therefore is not limited by data movement to the device but by the narrower per-thread issue width and lower aggregate bandwidth realised through the CPU memory subsystem.

The CPU result is useful as a fallback ceiling: a deployment that loses access to the GPU—for example because Metal is being used by a foreground graphics application—can still produce 20 tokens/sec on TinyLlama-1.1B, which remains above the human reading rate of ~5 tokens/sec.

XI. LIMITATIONS

We discuss several limitations of the study to clarify the scope of the conclusions.

Single hardware platform. All measurements are taken on a single Apple Silicon machine. The relative breakdown—MLP and projections dominate, attention is small at short context, RMSNorm is dispatch-bound—should generalize to other architectures whose GPUs are similarly memory-bandwidth-limited at decode-time arithmetic intensities (NVIDIA consumer GPUs, AMD RDNA mobile parts, Qualcomm Adreno). Absolute numbers will differ. We have not verified the quantitative scaling on other hardware in this paper; doing so is explicit future work.

Hook-induced overhead. The synchronization barriers introduced by our PyTorch forward hooks inflate absolute step latency by approximately 30–40% versus an un-instrumented model. The *relative* percentages reported in Table IV remain valid because the overhead is uniform across modules, but readers should not interpret the 21 ms median as the ceiling of MPS performance for TinyLlama-1.1B; an un-hooked, **torch.compile**-d deployment would run several milliseconds faster per token.

Limited model coverage. We measure two model sizes (1.1B and 3.4B). Larger models (LLaMA-2-7B, Mistral-7B, Phi-3) would not fit in the 17.2 GB unified-memory budget at float16 alongside the OS working set, and would therefore require quantization simply to load. Extending the study to such models is contingent on a larger-memory machine or a quantized-only configuration; we discuss this in the future work.

No quantization quality evaluation. The INT8 measurements (Section IX-D) report latency only. We rely on published perplexity results [10, 8] to characterize quality and have not

reproduced these on TinyLlama-1.1B. A faithful evaluation would run a held-out perplexity benchmark (WikiText-2, C4) or downstream task accuracy (HellaSwag, ARC) on the quantized model and report the delta.

Batch size fixed at 1. All experiments use batch size 1, matching the single-user interactive setting that motivated the work. At larger batch sizes the arithmetic intensity of decode steps rises linearly: per-token latency would still grow, but per-token throughput across the batch can multiply by an order of magnitude. The bandwidth-bound conclusions of this paper apply specifically to the batch-1 regime; multi-tenant serving scenarios behave differently and are well-characterized by prior work [2, 13].

No measurement of quantization accuracy on MPS. Our INT8 result uses the bitsandbytes mixed-precision LLM.int8() algorithm. Because the underlying INT8 GEMM kernel on MPS is not yet hardware-accelerated in the same way as on CUDA, the reported speedup reflects the bandwidth saving of INT8 weight storage but *not* any INT8-tensor-core compute speedup. On hardware with INT8 matmul acceleration (NVIDIA Ampere or newer, modern Apple Neural Engine), the end-to-end gain would likely be larger.

XII. CONCLUSION AND FUTURE WORK

We have presented a systematic, four-phase empirical study of autoregressive token-generation latency in LLaMA-family models on Apple Silicon. Our benchmark harness achieves 3.7% trial-to-trial coefficient of variation, validating the reproducibility of MPS execution. Component-level decomposition via PyTorch forward hooks reveals that MLP feed-forward blocks (28.2%) and linear projections (31.5%) together dominate decode-step time—not attention—at typical context lengths.

Scaling analysis confirms three architectural predictions of the roofline model: (1) latency scales linearly with KV-cache size, reflecting bandwidth-limited attention reads; (2) latency scales proportionally with model parameter count, reflecting weight streaming cost; and (3) float16 delivers a 1.89× speedup over float32, consistent with 2× bandwidth reduction. The primary bottleneck—weight streaming in bandwidth-bound GEMVs—is directly addressable through post-training quantization, which we project to reduce decode latency by 25–40% with minimal quality degradation.

These findings have broader implications for LLM deployment on edge devices with unified memory. As LLMs move from datacenter GPUs to consumer hardware, the memory-bandwidth bottleneck becomes more acute. Techniques that reduce per-token memory traffic—quantization, KV-cache compression, fused kernels—are the highest-leverage optimizations in this regime.

A. Future Work

Several directions extend this study and would strengthen the optimization claims.

INT4 GPTQ quantization with quality evaluation. The strongest remaining latency lever predicted by our roofline analysis is INT4 weight-only quantization via GPTQ [8]. Implementing this on MPS requires either porting the auto-gptq dequantization kernel or composing fp16 dequantize with the existing matmul kernel. A complete evaluation would report end-to-end decode latency *and* perplexity on WikiText-2 or C4 to verify that the projected ~1.4–1.6× speedup is achievable without unacceptable quality loss.

FlashAttention on MPS. The 8.8% attention-core time grows linearly with context length and becomes the dominant component beyond ~4K tokens. A Metal-native FlashAttention port would fuse the attention QKV-matmul-softmax-output sequence into a single kernel, eliminating intermediate HBM round-trips. The PyTorch team has prototyped Metal kernels for scaled dot-product attention; once these stabilize we would re-run our scaling study with FlashAttention enabled and quantify the impact on long-context decode.

Cross-platform validation on NVIDIA GPUs. Reproducing the hook-based decomposition on an NVIDIA Ampere or Hopper GPU would test whether the relative breakdown is hardware-invariant or whether the specific dispatch-overhead profile of MPS exaggerates the LayerNorm and framework-overhead components. We anticipate the relative ordering will hold but the absolute LayerNorm fraction will shrink because CUDA kernel-launch overhead is lower than the Metal command-encoder cost.

Larger models. Extending the parameter-scaling study to LLaMA-2-7B and 13B (loaded in INT8 to fit the 17.2 GB memory budget) would test the linear-scaling prediction beyond the 3.4B endpoint measured here. We expect TTFT and per-token latency to continue scaling with parameter count, but absolute throughput on a 7B model in INT8 should remain interactive (■10 tokens/sec).

Speculative decoding. Speculative decoding pairs a small draft model with the target model to amortize the per-step cost of the large model across several candidate tokens. Combining speculative decoding with our INT8 baseline could double effective throughput at no quality cost. Measuring the interaction between draft-model size, acceptance rate, and total throughput on Apple Silicon is an attractive next study because the unified-memory architecture permits both models to share weights with no host-device transfers.

REFERENCES

[1] J. Nielsen, *Usability Engineering*. San Francisco, CA: Morgan Kaufmann, 1993.

[2] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] G. Gerganov, "llama.cpp: Efficient LLaMA inference in C/C++," *GitHub repository*, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[5] H. Touvron *et al.*, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[6] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[7] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[9] Z. Liu *et al.*, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," *arXiv preprint arXiv:2402.02750*, 2024.

[10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[11] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023.

[12] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[13] Y. Sheng *et al.*, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023.

[14] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2023.

[15] PyTorch Team, "Introducing Accelerated PyTorch Training on Mac," *PyTorch Blog*, 2022. [Online]. Available: https://pytorch.org/blog/introducing-accelerated-pytorch-training-on-mac/